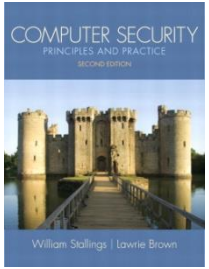
אבטחת סייבר

(אבטחת מידע/מחשבים/מערכות ממוחשבות/...)



המצגת הינה לשימוש פנימי בלבד

אין לצלם או לעשות שימוש שלא במסגרת הקורס

בפעמים הקודמות

- דיברנו על "פרימיטיבים" של הצפנה ואימות – אבני בניין שישמשו אותנו לפי הצורך
  - הצפנה סימטרית/אסימטרית (AES,RSA)
  - אימות הודעות (MAC, חתימה דיגיטלית)
  - החלפת מפתחות (תעודות)
- דיברנו על בקרת כניסה
  - (פירוט בהמשך)

היום

- תזכורת
- דוגמה לשימוש באבני בנייך: TLS/SSL
- פרוטוקול קרברוס
- בקרת גישה

תזכורת

מפתח פומבי

- ידע שונה – בעל המפתח יודע יותר
- מפתח פומבי
- מפתח פרטי
- מבוססים על בעיות מתמטיות קשות

פרימיטיבים סימטריים

- ידע זהה לשני המשתתפים
- מפתח משותף
- או ללא מפתח
- מבוססים על מבנה שנראה קשה

עקרון קרכהופס

- צופן ידוע לכל
- מפתח סודי

צופן	
$M=D_K(C)$	$C=E_K(M)$

אופני תפעול

- ECB
- CBC
- CFB
- OFB
- CTR

תזכורת

פרימיטיבים סימטריים

- ידע זהה לשני המשתתפים
- מפתח משותף
- או ללא מפתח
- מבוססים על מבנה שנראה קשה

מפתח פומבי

- ידע שונה – בעל המפתח יודע יותר
- מפתח פומבי
- מפתח פרטי
- מבוססים על בעיות מתמטיות קשות

צפנים	מהירות	הנחות	ניהול מפתחות
סימטריים	✓ מהירים	הנחות אד-הוק	מסובך
אסימטריים	איטיים	בעיות מתמטיות קשות	✓ פשוט יותר

החלפת מפתחות אסימטרית

הצפנה סימטרית

תזכורת

פרימיטיבים סימטריים

- ידע זהה לשני המשתתפים
- מפתח משותף
- או ללא מפתח
- מבוססים על מבנה שנראה קשה

מפתח פומבי

- ידע שונה – בעל המפתח יודע יותר
- מפתח פומבי
- מפתח פרטי
- מבוססים על בעיות מתמטיות קשות

צפנים	מהירות	הנחות	ניהול מפתחות
סימטריים	✓ מהירים	הנחות אד-הוק	מסובך
אסימטריים	איטיים	בעיות מתמטיות קשות	✓ פשוט יותר

החלפת מפתחות אסימטרית

הצפנה סימטרית

	אימות השולח	שלמות	אי-הכחשה
חתימות דיגיטליות	☑	☑	☑
פונקציות אימות	☑	☑	×

תזכורת

מפתח פומבי
פרימיטיבים סימטריים
חתימות
RSA
DSA
ECDSA
El-Gamal
החלפת מפתחות
Diffie-Hellman
צפני מפתח פומבי
RSA
פונקציות אימות
HMAC-SHA-1
HMAC-SHA-2
HMAC-SHA-3
CBC-MAC
פונקציות תמצות
SHA-1
SHA-2
SHA-3
צפנים סימטריים
DES
AES
RC4
One-Time Pad
+ אלגוריתם

אזהרות

אין להשתמש בצופן ללא אימות של כתב הסתר!
- במיוחד כאשר תוקף אקטיבי עלול להשתתף בפרוטוקול
- ובפרוטוקולים מורכבים, כגון פרוטוקלי רשתות מחשבים

אזהרות

אין להשתמש בצופן ללא אימות של כתב הסתר!
- במיוחד כאשר תוקף אקטיבי עלול להשתתף בפרוטוקול
- ובפרוטוקולים מורכבים, כגון פרוטוקלי רשתות מחשבים

אין להשתמש באותו מפתח להצפנה ב-CBC ואימות ב-CBC-MAC

אזהרות

אין להשתמש בצופן ללא אימות של כתב הסתר!
- במיוחד כאשר תוקף אקטיבי עלול להשתתף בפרוטוקול
- ובפרוטוקולים מורכבים, כגון פרוטוקלי רשתות מחשבים

אין להשתמש באותו מפתח להצפנה ב-CBC ואימות ב-CBC-MAC

אין להשתמש ב-RSA ללא PADDING (למשל OAEP)

אזהרות

אין להשתמש בצופן ללא אימות של כתב הסתר!
- במיוחד כאשר תוקף אקטיבי עלול להשתתף בפרוטוקול
- ובפרוטוקולים מורכבים, כגון פרוטוקלי רשתות מחשבים

אין להשתמש באותו מפתח להצפנה ב-CBC ואימות ב-CBC-MAC

אין להשתמש ב-RSA ללא PADDING (למשל OAEP)

אין לחתום על ההודעה המקורית (אלא על HASH שלה)

אזהרות

אין להשתמש בצופן ללא אימות של כתב הסתר!
- במיוחד כאשר תוקף אקטיבי עלול להשתתף בפרוטוקול
- ובפרוטוקולים מורכבים, כגון פרוטוקלי רשתות מחשבים

אין להשתמש באותו מפתח להצפנה ב-CBC ואימות ב-CBC-MAC

אין להשתמש ב-RSA ללא PADDING (למשל OAEP)

אין לחתום על ההודעה המקורית (אלא על HASH שלה)

אין להשתמש באותו מפתח להצפנה ב-RSA וחתימה ב-RSA

Basic Comprehension Survey

1. Bob encrypts a message with **symmetric key**, and sends it to Alice, who owns this key as well, what did he achieve?
2. Bob encrypts a message with his **private key**, and sends it to Alice, what did he achieve?
3. Bob encrypts a message with his **public key**, and sends it to Alice, what did he achieve?
4. Bob encrypts message with Alice's **public key**, and sends it to Alice, what did he achieve?
5. Bob encrypts message's hash code with his **private key**, and sends it to Alice with the message itself encrypted with Alice's **public key**, what did he achieve?
6. Bob encrypts message's hash code with Alice's **public key**, and sends it to Alice what did he achieve?

   a. Confidentiality
   b. Message authentication (integrity of content and time)
   c. Source authentication
   d. Nothing

SSL/TLS Handshake Protocol

- is used before any application data are transmitted
- allows server & client to:
  - authenticate each other
  - to negotiate encryption & MAC algorithms
  - to negotiate cryptographic keys to be used
- comprises a series of messages in phases
  - Establish Security Capabilities
  - Server Authentication and Key Exchange
  - Client Authentication and Key Exchange
  - Finish

SSL/TLS
תאור סכמטי

Browser sends "Hello" and requests a secure session from web server

Web server responds by sending over its server certificate, containing information about the site, Certificate Authority, etc. and a public key

Cipher suites example:
TLS_RSA_WITH_AES_256_CBC_SHA256
TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384

After the browser verifies the certificate, it sends a one-time "session" key encrypted with the server's public key *

*or use DHE

Server decrypts the one-time "session" key using its private key

A secure connection has been established. The padlock icon appears in the browser.

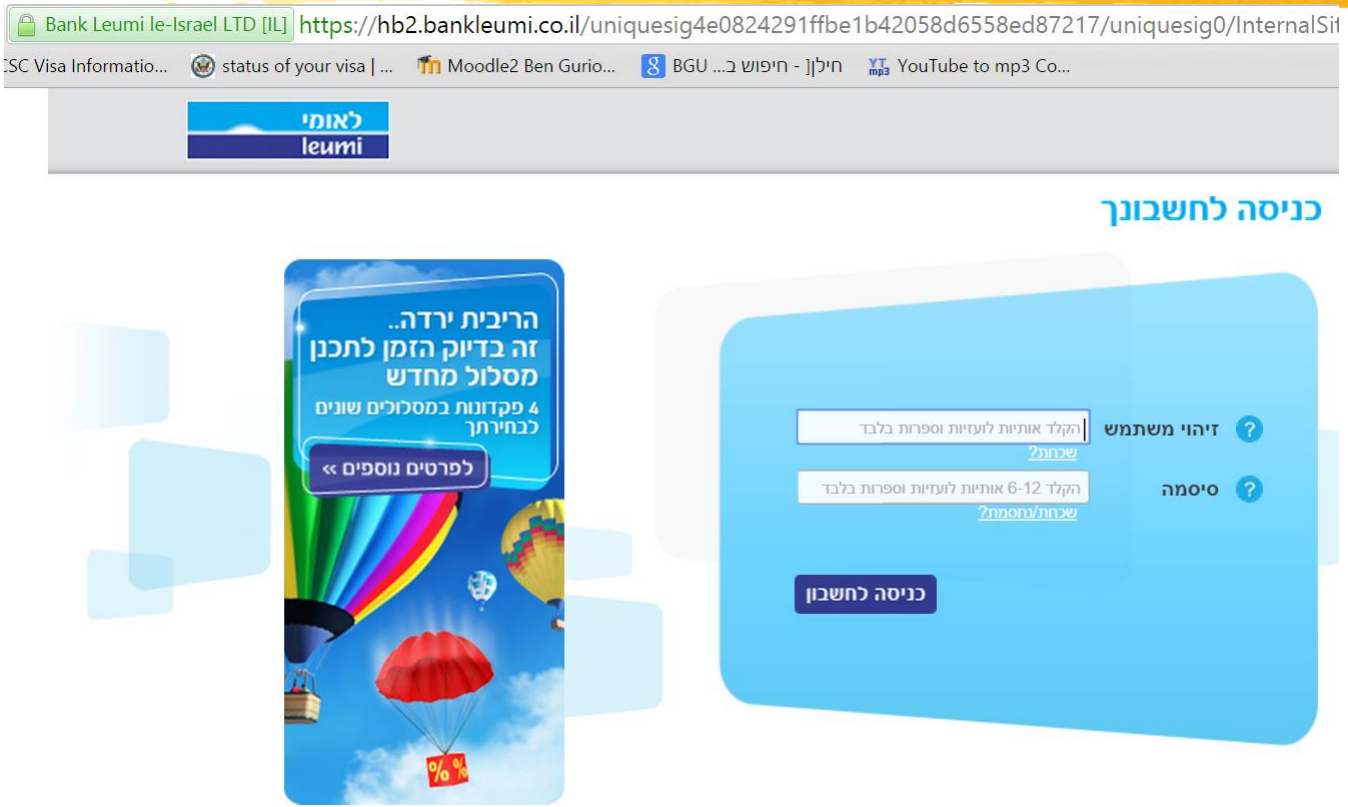
rest of session is encrypted with session key

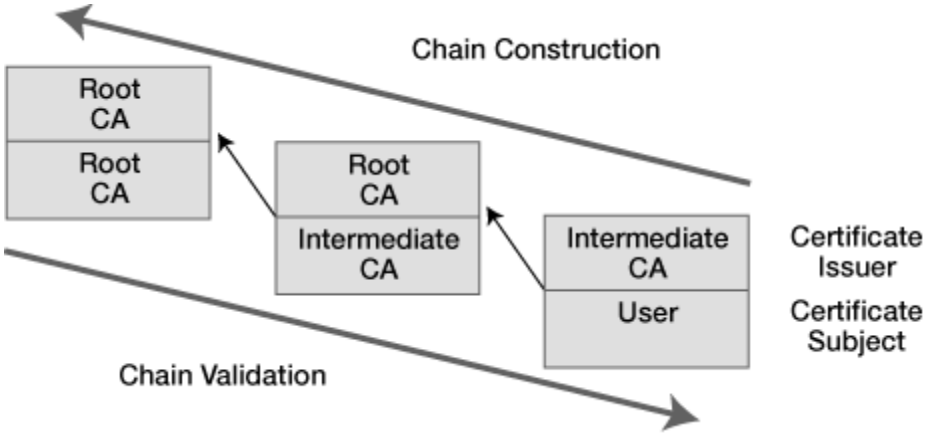
HTTPS (HTTP over SSL/TLS)

- Combination of HTTP and SSL/TLS to implement secure communication between a Web browser and a Web server

- built into all modern Web browsers
  - URL addresses begin with https://
  - documented in RFC 2818, HTTP Over TLS

HTTPS (HTTP over SSL)

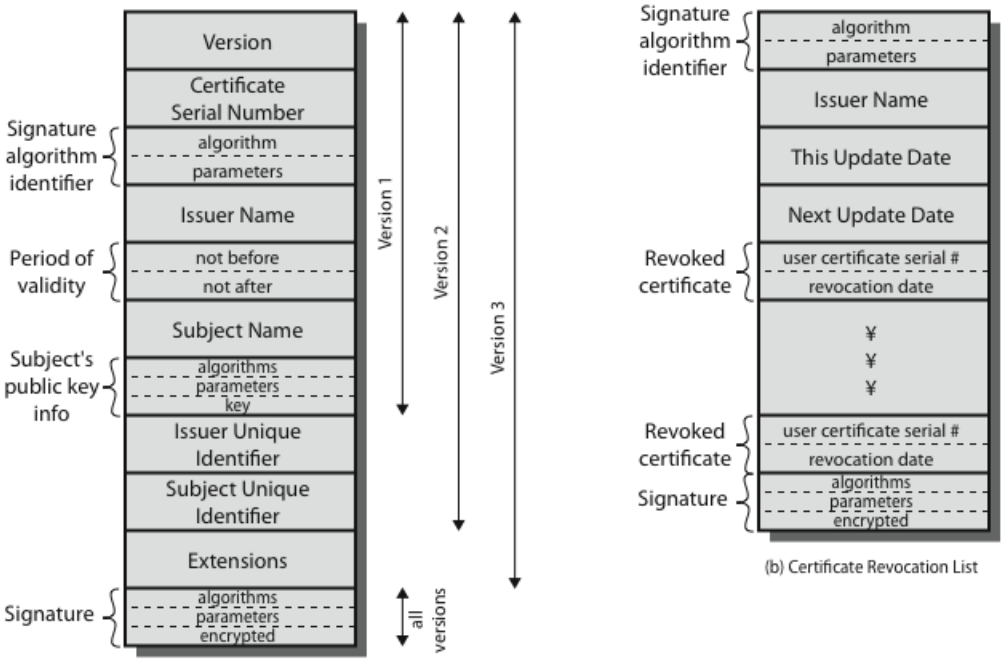



Chain Construction
Root
CA
Root
CA
Root
CA
Intermediate
CA
Intermediate
CA
User
Certificate
Issuer
Certificate
Subject
Chain Validation

X.509 Authentication Service

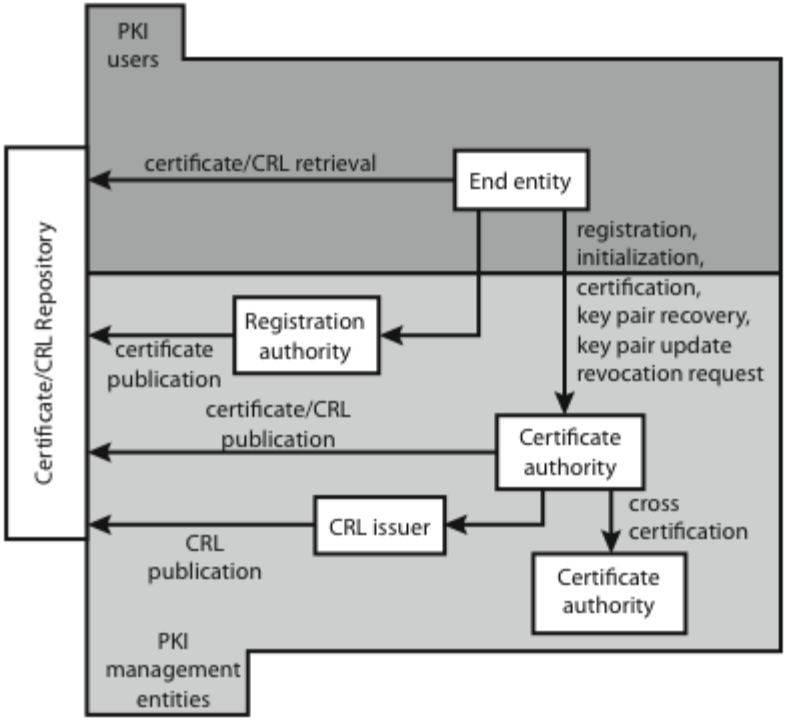
- universally accepted standard for formatting public-key certificates
- part of CCITT X.500 directory service standards
- uses public-key crypto & digital signatures

X.509 Certificates



(b) Certificate Revocation List

Public Key Infrastructure



PKIX Management

- functions:
  - registration
  - initialization
  - certification
  - key pair recovery
  - key pair update
  - revocation request
  - cross certification
- protocols: CMP, CMC

בקרת כניסה/אימות זיהוי בפעם הקודמת

- סיסמאות ואחסונם
  - התקפות מילון, SALT,...
- כרטיסים חכמים
  - אתגר-תגובה
- ביומטריה
  - שיקולי עלות-תועלת
  - False-positive,...
- Multi factor authentication
- סיסמאות גרפיות

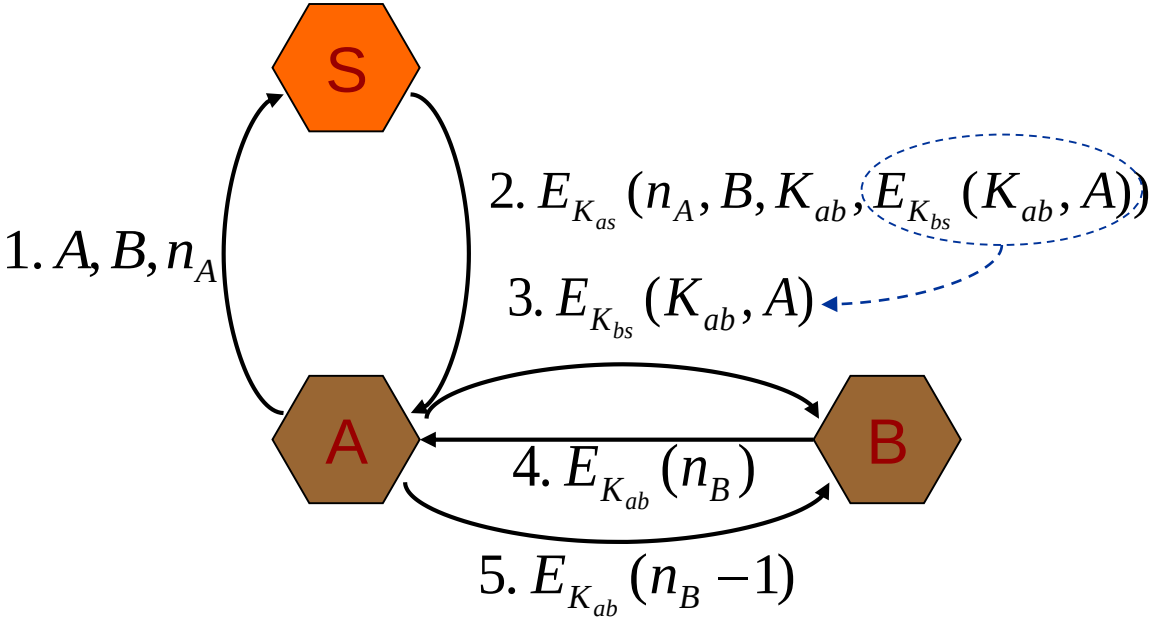
היום

- בקרת כניסה – המשך
  - פרוטוקול אימות זיהוי הדדי
  - KERBEROS
- בקרת גישה (Access Control)

Mutual Authentication

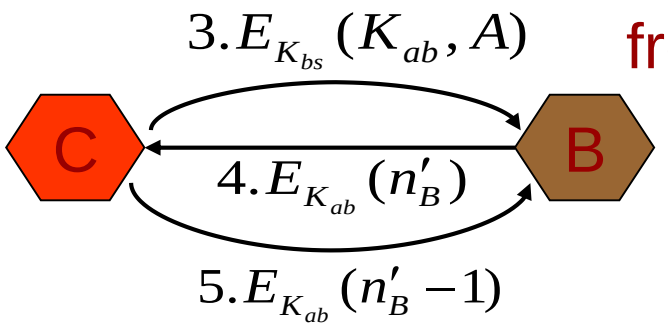
- KDC
- Shared key with KDC
- Time stamp
- Nonce
- Challenge-Response
- || means concatenation

Needham-Schroeder Protocol



Denning-Sacco Attack

- The NS protocol achieves its goals under the (standard) assumption that the long term keys K_{as} and K_{bs} are not compromised.
- Denning & Sacco discovered an attack where the adversary *C* impersonates *A* re-using a compromised session key K_{ab}:



from original protocol run

Denning's Protocol

1. $A \rightarrow KDC$: $ID_A || ID_B$
2. $KDC \rightarrow A$: $E_{k_a}[K_S || ID_B || T || E_{K_b}[K_S || ID_A || T]]$
3. $A \rightarrow B$: $E_{k_b}[K_S || ID_A || T]$
4. $B \rightarrow A$: $E_{k_s}[N_1]$
5. $A \rightarrow B$: $E_{k_s}[f(N_1)]$

- Use of time (T) prevents replaying old messages
- Requires synchronization of clocks

Kerberos

Kerberos

- trusted key server system from MIT
- provides centralised private-key third-party authentication in a distributed network
  - allows users access to services distributed through network
  - without needing to trust all workstations
  - rather all trust a central authentication server
- two versions in use: 4 & 5

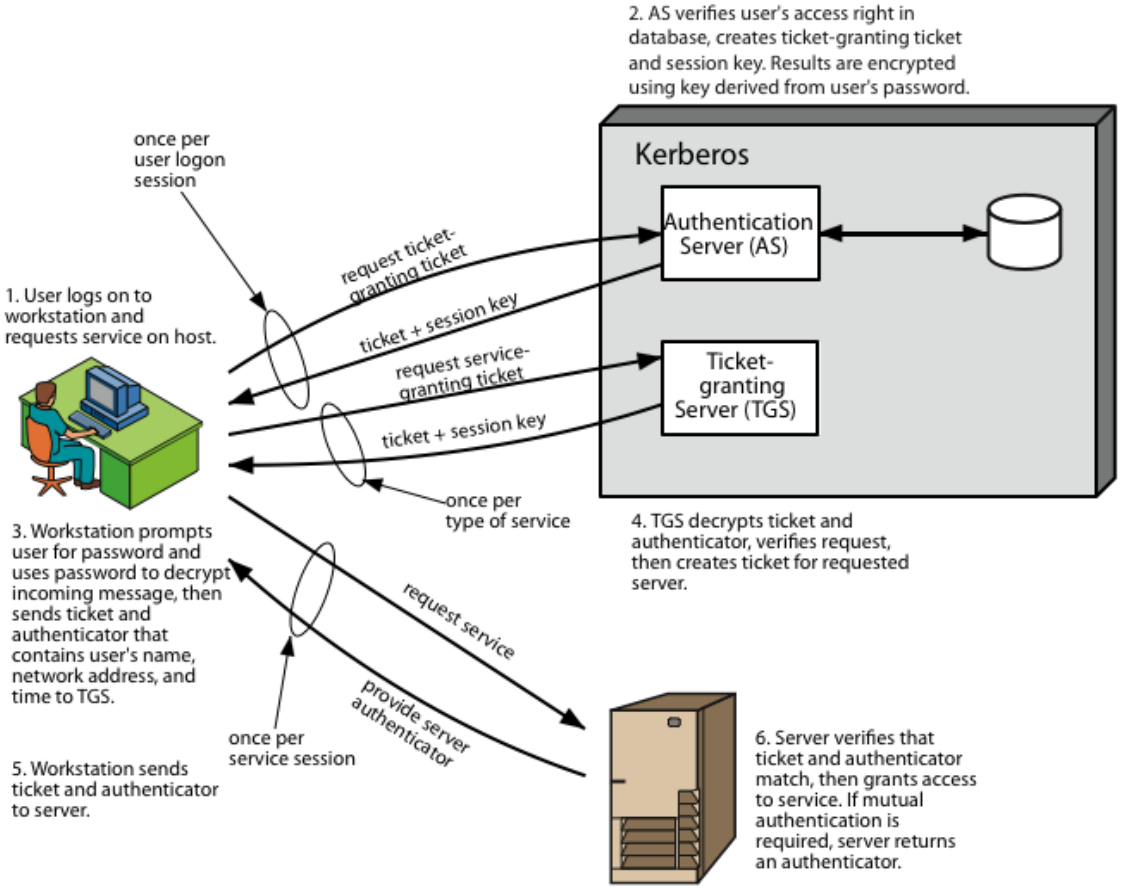
Kerberos - Overview

- A secured, centralized network authentication protocol (N users and M servers/services)
- Providing protection against replay, password guessing and session hijacking
- Single-Sign-On (SSO)
- Based on symmetric cryptography
- Developed as part of Project Athena at MIT (late 1980s)
- Used in many systems, e.g., Windows 2000 and later as default authentication protocol

Kerberos – Overview and Components

- Goal:
  - Allow secured and efficient work of many clients with many servers and systems usually within organization
  - Allow single sign on (SSO)
- Key Distribution Center (KDC)
- Users Database
- Tickets
- Kerberos server share secret key with each one of the Servers
  - distributed physically and securely.
- User X logs in to the station with his password only once (SSO)
- User receive tickets
  - used to prove authentication to each one of the desired servers for the daily work

Kerberos - Overview



Kerberose Version 4

(a) Authentication Service Exchange:
to obtain ticket-granting ticket

(1) → C AS: $ID_C || ID_{tgs} || TS_1$

(2) → AS C:
$E_{k_c}[ID_C || K_{c.tgs} || ID_{tgs} || TS_2 || Lifetime_2 || Ticket_{tgs}]$

$Ticket_{tgs} = E_{K_{tgs}}[K_{c.tgs} || ID_C || AD_C || ID_{tgs} || TS_2 || Lifetime_2]$

(b) Ticket-Granting Service Service Exchange:
to obtain service-granting ticket

(3) → C TGS: $ID_V || Ticket_{tgs} || Authenticator_C$

(4) TGS C: $E_{k_{c.tgs}}[K_{c.V} || ID_V || TS_4 || Ticket_V]$

$Ticket_{tgs} = E_{K_{tgs}}[K_{c.tgs} || ID_C || AD_C || ID_{tgs} || TS_2 || Lifetime_2]$

$Ticket_V = E_{K_V}[K_{c.V} || ID_C || AD_C || ID_V || TS_4 || Lifetime_4]$

$Authenticator_C = E_{k_{c.tgs}}[ID_C || AD_C || TS_3]$

Kerberose Version 4, cont.

(c) Client/Server Authentication Exchange:
to obtain service

(5) C -> V: $Ticket_V || Authenticator_C$

(6) V -> C: $E_{k_{C.V}}[TS_5+1]$ (for mutual authentication)

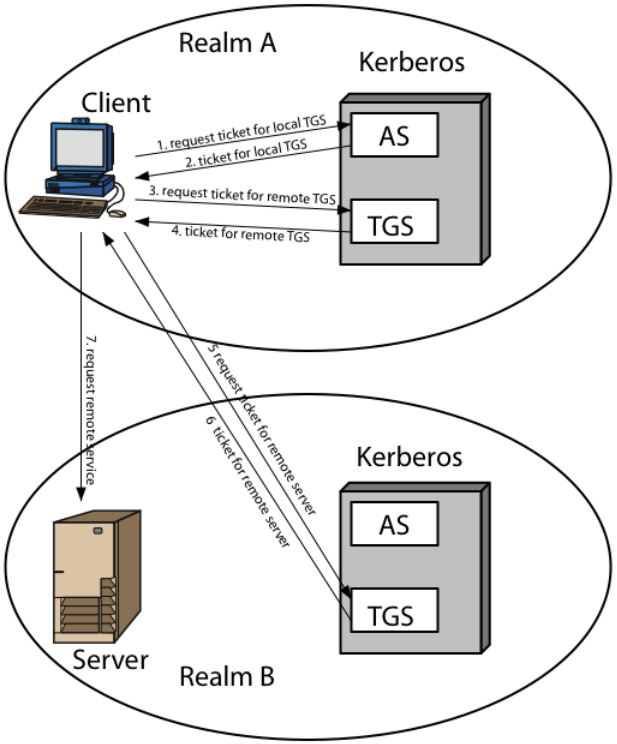
$Ticket_V = E_{K_V}[K_{C.V} || ID_C || AD_C || ID_V || TS_4 || Lifetime_4]$

$Authenticator_C = E_{k_{C.V}}[ID_C || AD_C || TS_5]$

Kerberos Realms

- a Kerberos environment consists of:
  - a Kerberos server
  - a number of clients, all registered with server
  - application servers, sharing keys with server
- this is termed a realm
  - typically a single administrative domain
- if have multiple realms, their Kerberos servers must share keys and trust

Kerberos Realms



Kerberos Version 5

- Kerberos v5 is the most widely used version
- Developed in mid 1990's
  - specified as Internet standard RFC 1510
- provides improvements over v4
  - addresses environmental shortcomings
    - encryption alg, network protocol, byte order, ticket lifetime, authentication forwarding, inter-realm auth
  - and technical deficiencies
    - double encryption, non-std mode of use, session keys, password attacks

Kerberos - Advantages

- TGT (Ticket Granting Ticket) with the secret key of the user – thus user authentications is guaranteed.
- AS provides the client a TGT (Ticket Granting Ticket) that is good for TGS that can provide a key for every server.
- Using the Granting Ticket, the client can ask additional tickets for additional servers, thus:
  - We don't bother the <u>user</u> with additional PSW requests
- Time-Stamp strengths the protocol against:
  - replay attack.
  - Reuse of tickets – they expire after predefined period (e.g 4 hours)
- Secured since integrates third party authentication entity:
  - the tickets cannot be altered by users since encrypted with secret key of AS and TGS.
  - MITM is not feasible.

Kerberos - Drawback

- Single point of failure (AS)
- Requires online trusted third party: Kerberos server
- Security depends on tight clock synchronization
- Secret keys are temporarily stored on the users' workstation
- Password file can be exposed; dictionary attack
  - Even without it, offline attack possible – how?

Access Control

Access Control

- ITU-T (International Telecommunication Union)
- Recommendation X.800 defines access control as follows:

- "The prevention of unauthorized use of a resource, including the prevention of use of a resource in an unauthorized manner."

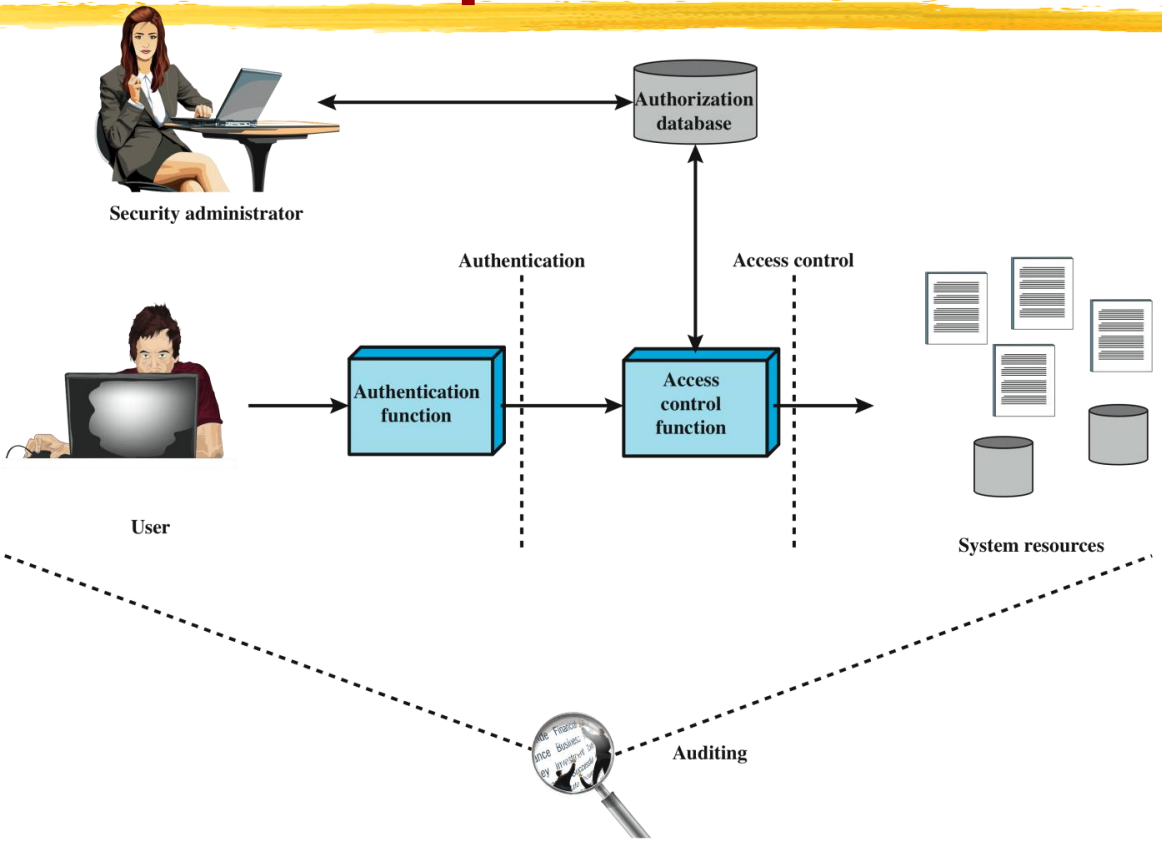
Access Control Principles

- **Authentication:**
  - Verification that the claimed identity of a user or other system entity are valid.

- **Authorization:**
  - The granting of a right or permission to a system entity to access a system resource.
  - This function determines who is trusted for a given purpose.

- **Audit:**
  - An independent review and examination of system records and activities
  - test for adequacy of system controls, to ensure compliance with established policy and operational procedures,
  - detect breaches in security
  - recommend any indicated changes in control, policy and procedures.

Access Control Principles



Security Policies and models

A security model is a formal definition of a security policy

Example of university policies

- An instructor can look at all the information about the course he is teaching.
- An instructor can change the grades of the students in the course he is teaching
- A student may look at her grades in a course she is taking
- The department head can add/delete course offerings
- The department head can add/delete students from course offerings
- Faculty members can look at information about themselves

Examples for Authorization specifications

- John has a Faculty type of access to course: Operating Systems
- Bill has a student type of access to course; Data Structures
- Mary (secretary) has a Read/write access to all grades files

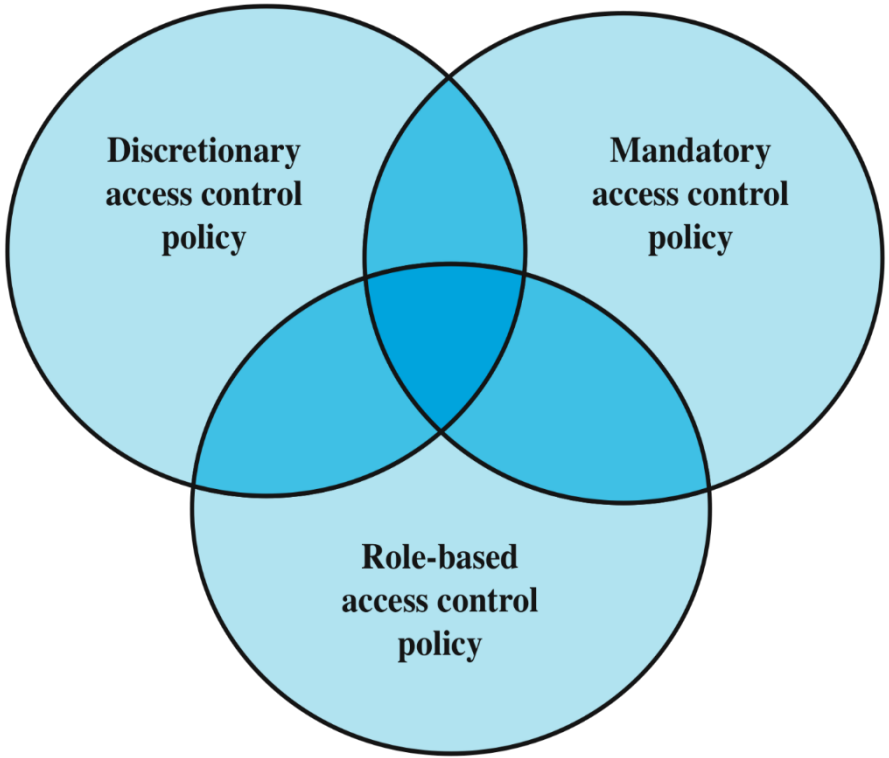
Why Security Models?

- When we have implemented a security policy, do we know that it will (and can) be enforced?
- E.g., if policies get too intricate, contradicting rules may apply to a given access request.
- To prove that a policy will be enforced, the policy has to be formally specified.

Multiple Access Control Policies

DAC

Entity can enable another entity to access some resources



MAC

Entity cannot enable another entity to access some resources

RBAC -(Roles and rules defined in the sytem controls the access

Access Control Requirements

- Reliable input (authenticated user, IP address…)
- Support for fine and coarse specifications
- Least privilege
- Separation of duty
- Open and closed policies
- Policy combinations and conflict resolution
- Administrative policies
- Dual control
- Authorization creep (e.g. when employee is Terminated without removing their old access)
- Security domain – all resources and users that are working under the same security policy
- Single-Sign-On (SSO)

Access Control Basic Elements

- subject – entity capable of accessing objects
  - User or application gains access by process
  - typically held accountable for the actions they initiate
- object – resource to which access is controlled
  - entity used to contain and/or receive information
  - protection depends on the environment in which access control operates
- access right – describes the way in which a subject may access an object
  - e.g. read, write, execute, delete, create, search

Discretionary Access Control Model (DAC)

Are stated in terms of a set of

- security objects,
- security subjects, and
- access privileges

Basic primitives:

- Users can protect the data they "own".
- The owner may grant access to others.
- The owner may define the type of access (read, write execute, ...) given to others.
- Granting and revoking of access permission is under the discretion of the users themselves.

Discretionary Access Control Matrix

- DAC is often provided using an access matrix
  - one dimension consists of identified subjects that may attempt data access to the resources
  - the other dimension lists the objects that may be accessed
- Each entry in the matrix indicates the access rights of a particular subject for a particular object

The Access Matrix Model

- **Subjects**
  - users, groups, applications, transactions…
- **Objects**
  - Files, programs, databases, relations, URLs…
- **Access-types**
  - Read, write, create, copy, delete, execute, kill…
  - **May differ in different implementations**
- **Authorization commands**
  - enter, remove, transfer…
- **Authorizers**
  - Owners, users, administrators…

Access Matrix

		OBJECTS			
		File 1	File 2	File 3	File 4
SUBJECTS	User A	Own Read Write		Own Read Write	
	User B	Read	Own Read Write	Write	Read
	User C	Read Write	Read		Own Read Write

(a) Access matrix

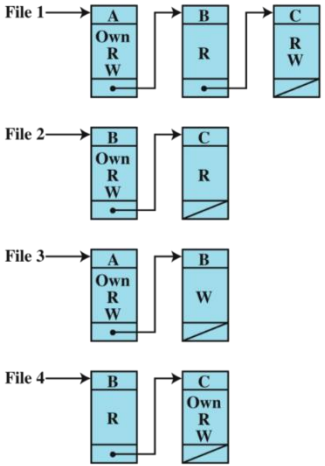
Usually Sparse Matrix, why ?

Example of Access Control Structures

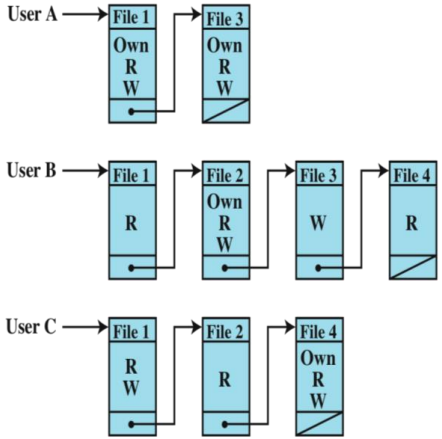
- (b) ACLs = Access Control Lists
  - Per columns = per object
- (c) Capability ticket
  - Per row= per subject (user\system)
  - Can also contain default\public entry

	OBJECTS			
	File 1	File 2	File 3	File 4
User A	Own Read Write		Own Read Write	
User B	Read	Own Read Write	Write	Read
User C	Read Write	Read		Own Read Write

(a) Access matrix



(b) Access control lists for files of part (a)



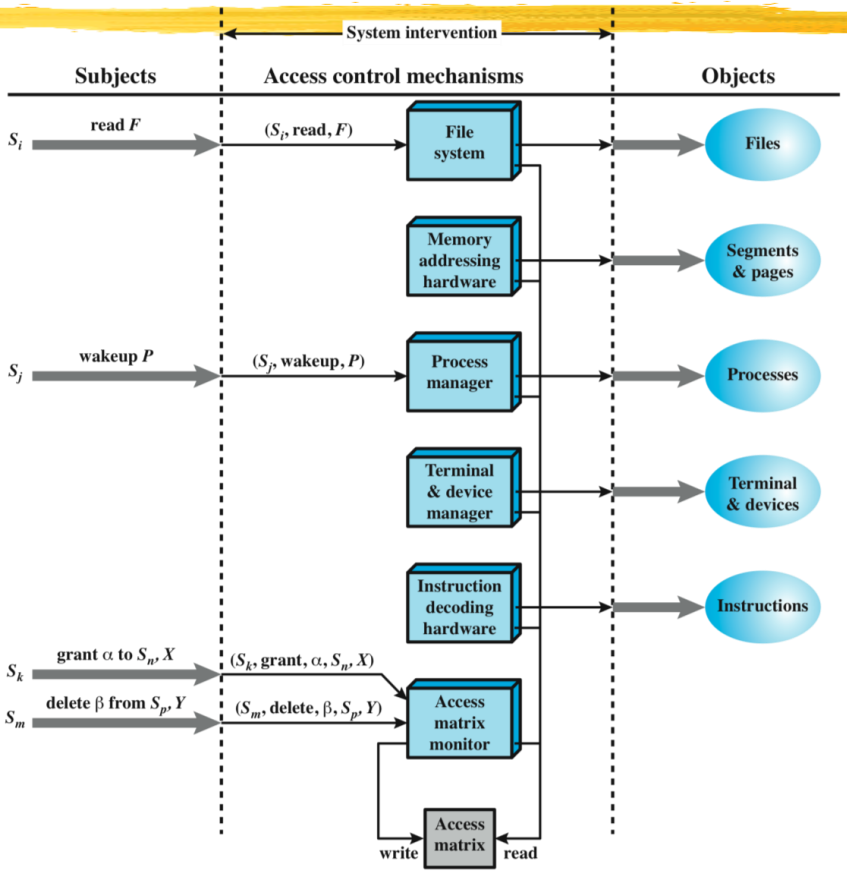
(c) Capability lists for files of part (a)

Extended Access Control Matrix

		OBJECTS								
		subjects			files		processes		disk drives	
		S_1	S_2	S_3	F_1	F_2	P_1	P_2	D_1	D_2
SUBJECTS	S_1	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
	S_2		control		write *	execute			owner	seek *
	S_3			control		write	stop			

* - copy flag set

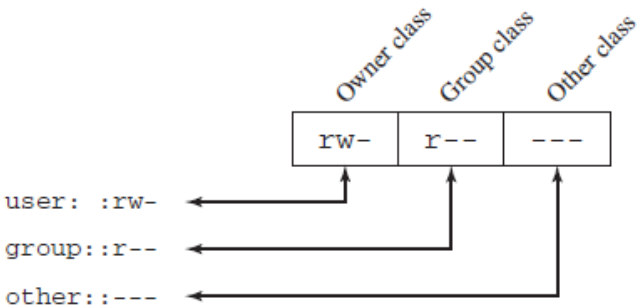
Access Control Function



Access Control System Commands

Rule	Command (by S_o)	Authorization	Operation
R1	**transfer** $\begin{Bmatrix} \alpha * \\ \alpha \end{Bmatrix}$ **to** S, X	'α*' in $A[S_o, X]$	store $\begin{Bmatrix} \alpha * \\ \alpha \end{Bmatrix}$ in $A[S, X]$
R2	**grant** $\begin{Bmatrix} \alpha * \\ \alpha \end{Bmatrix}$ **to** S, X	'owner' in $A[S_o, X]$	store $\begin{Bmatrix} \alpha * \\ \alpha \end{Bmatrix}$ in $A[S, X]$
R3	**delete** α **from** S, X	'control' in $A[S_o, S]$ or 'owner' in $A[S_o, X]$	delete α from $A[S, X]$
R4	$w \leftarrow$ **read** S, X	'control' in $A[S_o, S]$ or 'owner' in $A[S_o, X]$	copy $A[S, X]$ into w
R5	**create object** X	None	add column for X to A; store 'owner' in $A[S_o, X]$
R6	**destroy object** X	'owner' in $A[S_o, X]$	delete column for X from A
R7	**create subject** S	none	add row for S to A; execute **create object** S; store 'control' in $A[S, S]$
R8	**destroy subject** S	'owner' in $A[S_o, S]$	delete row for S from A; execute **destroy object** S

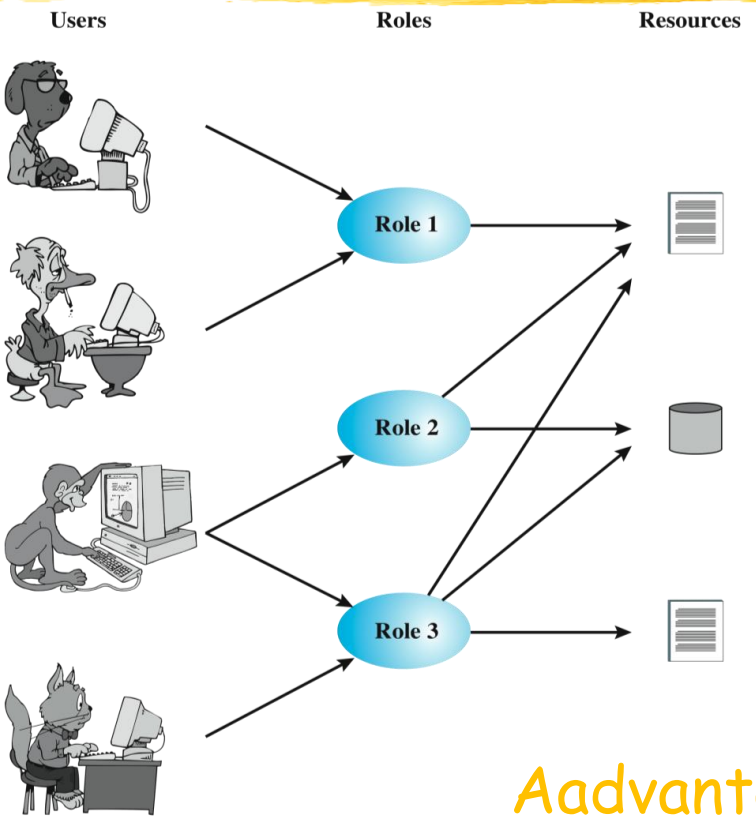
Example: UNIX File Access Control



(a) Traditional UNIX approach (minimal access control list)

- Inode (Index node) = control structure needed for OS for particular file. Contains:
- Each user has unique user identification number (user ID)
- Each user is a member of a primary group identified by a group ID
- Each user belongs to a specific group
- 12 protection bits
  - specify read, write, and execute permission for the owner of the file, members of the group and all other users
- First 9 bits include authorization to USER,GROUP and other.
- the owner ID, group ID, and protection bits are part of the file's inode

Role-Based Access Control (RBAC)



Aadvantage over DAC ?

RBAC - Access Control Matrix

	R_1	R_2	...	R_n
U_1	X			
U_2	X			
U_3		X		X
U_4				X
U_5				X
U_6				X
...				
U_m	X			

- Assigning Users to Roles:
  -
  -
  -

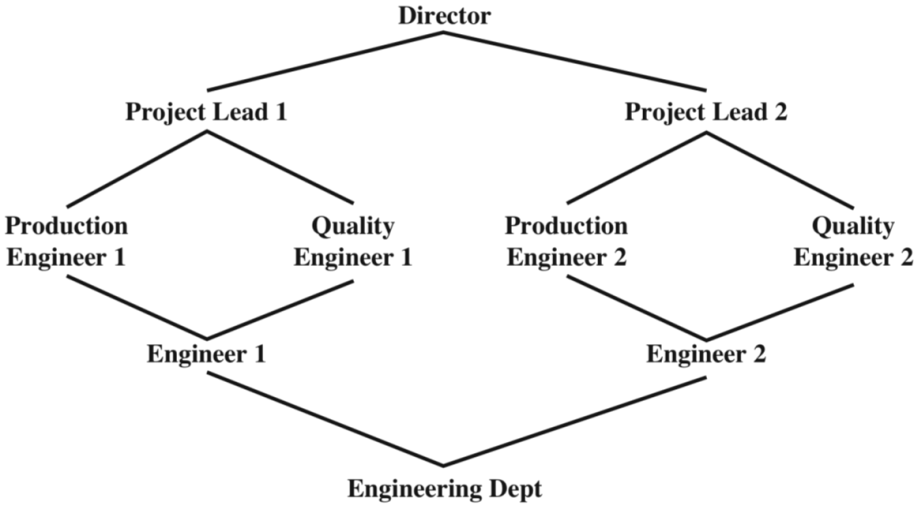
- Assigning Roles to Objects:
  -
  -

ROLES \ OBJECTS	R_1	R_2	R_n	F_1	F_1	P_1	P_2	D_1	D_2
R_1	control	owner	owner control	read *	read owner	wakeup	wakeup	seek	owner
R_2		control		write *	execute			owner	seek *
...									
R_n			control		write	stop			

Role-Based Models

- **RBAC0** – Users, Roles, Permissions, Sessions
- **RBAC1** – RBAC0 + Role-hierarchies
- **RBAC2** – RBAC0 + Constraints
- **RBAC3** – RBAC0 + Role-hierarchies + Constraints

Example of Role Hierarchy – RBAC1



The Information Flow Problem of DAC (I)

Users Rights

r: Alice; w: Alice

User: Alice

File A

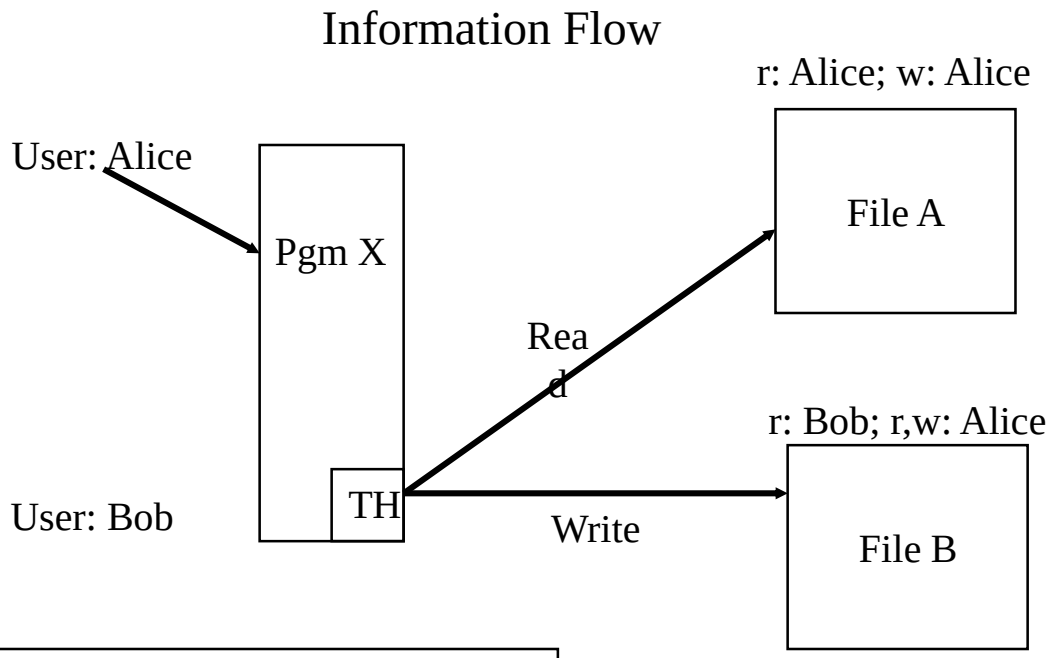
r: Bob; r,w: Alice

User: Bob

File B

User Bob cannot read the file A!

The Information Flow Problem of DAC (II)



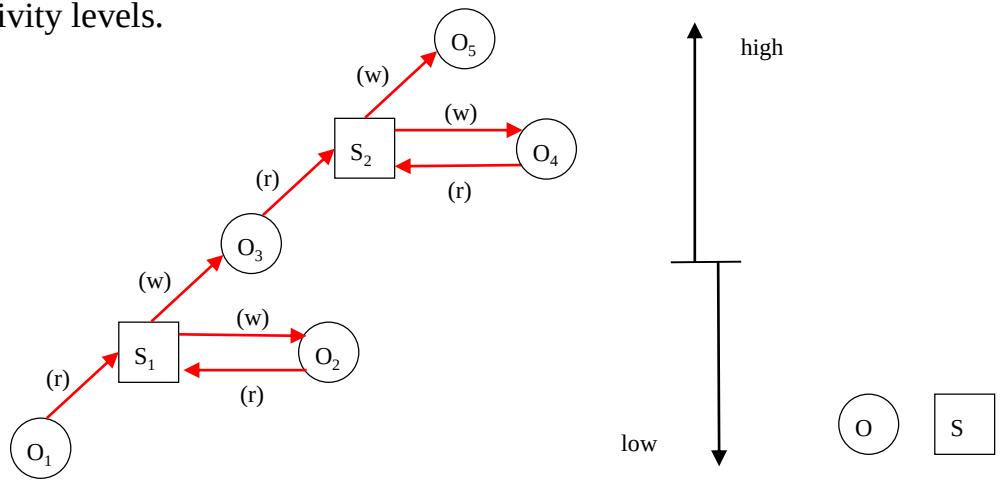
User Bob can read contents of the file A copied to file B

Bell-LaPadula (BLP) Confidentiality Model (1973)

- By David Bell and Leonard LaPadula
- Hierarchical
- Addresses the confidentiality of information in a system
- Mandatory access control
- Formalize the U.S. Department of Defense (DoD) multilevel security (MLS) policy; the basis for the Orange Book
- Simple security property – read capability only
  - A subject can read information at or below his level of secrecy, but cannot read any information above his level of secrecy (no read up)
- Star property – write capability only
  - A subject can write information at or above his level of secrecy, but cannot read any information above his level of secrecy (no write down)

Bell-LaPadula (BLP) Confidentiality Model (1973)

The *-property protects information from being ‚written-down‘ along the hierarchy of sensitivity levels.



‚Write‘ if no ‚read‘ to higher classified data!

Features

- A mandatory model (users are assigned to classifications by administrators)
- Need trusted processes to perform administrative functions
- Typically use data labels to enforce security

Problems of BLP

1. The need for a Trusted subject (to write non-secret info.)

Solution: **high water mark**! – get the sensitivity level of the highest level accessed object during the program execution so far…

2. The problem of covert channels.
3. No access-control when all subjects and all objects are at the same level.